



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 06

NOMBRE COMPLETO: Bailón Cruz Owen Isaac

N° de Cuenta: 320069688

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 12/04/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Aquí declaro las texturas y objetos que estare usando.

```
38 Texture brickTexture;
39 Texture dirtTexture;
40 Texture plainTexture;
41 Texture pisoTexture;
42 Texture dadoTexture;
43 Texture cocheTexture;
44 Texture logofiTexture;
45
46 Model Kitt_M;
47 Model Llanta_M;
48 Model Llanta_M2;
49 Model Llanta_M3;
50 Model Llanta_M4;
51 Model Parb;
52 Model Dado_M;
53
54 Skybox skybox;
55
```

Aquí modificó el Create Objects para poder crear un Octaedro Regular en vez de un cubo como estaba antes.

```
102 void CreateObjects()
103 {
104     unsigned int indices[] = {
105         0, 3, 1,
106         1, 3, 2,
107         2, 3, 0,
108         0, 1, 2
109     };
110
111     GLfloat vertices[] = {
112         // x      y      z      u      v      nx      ny      nz
113         -1.0f, -1.0f, -0.6f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f,
114         0.0f, -1.0f, 1.0f, 0.5f, 0.0f, 0.0f, 0.0f, 0.0f,
115         1.0f, -1.0f, -0.6f, 1.0f, 0.0f, 0.0f, 0.0f, 0.0f,
116         0.0f, 1.0f, 0.0f, 0.5f, 1.0f, 0.0f, 0.0f, 0.0f
117     };
118
119     unsigned int floorIndices[] = {
120         0, 2, 1,
121         1, 2, 3
122     };
123
124     GLfloat floorVertices[] = {
125         -10.0f, 0.0f, -10.0f, 0.0f, 0.0f, 0.0f, -1.0f, 0.0f,
126         10.0f, 0.0f, -10.0f, 10.0f, 0.0f, 0.0f, -1.0f, 0.0f,
127         -10.0f, 0.0f, 10.0f, 0.0f, 10.0f, 0.0f, -1.0f, 0.0f,
128         10.0f, 0.0f, 10.0f, 10.0f, 10.0f, 0.0f, -1.0f, 0.0f
129     };
130
131     unsigned int vegetationIndices[] = {
132         0, 1, 2,
133         0, 2, 3,
134         4, 5, 6,
135         4, 6, 7
136     };
137 }
```

Aquí modifico el código de mapeo del shader para el dado para poder hacer triángulos en vez de cuadrados como antes

```

194 GLfloat octa_vertices[] = {
195     // x      y      z      S      T      NX      NY      NZ
196
197     // MITAD SUPERIOR
198
199     // Cara 1
200     0.0f, 0.5f, 0.0f, 0.5f, 0.25f, 0.0f, 0.0f, 0.0f, // Punta Superior
201     -0.5f, 0.0f, 0.0f, 0.75f, 0.5f, 0.0f, 0.0f, 0.0f, // Punta Izquierda
202     0.0f, 0.0f, 0.5f, 0.25f, 0.5f, 0.0f, 0.0f, 0.0f, // Punta Derecha
203
204     // Cara 2
205     0.0f, 0.5f, 0.0f, 0.0f, 0.25f, 0.0f, 0.0f, 0.0f,
206     0.0f, 0.0f, 0.5f, 0.5f, 0.25f, 0.0f, 0.0f, 0.0f,
207     0.5f, 0.0f, 0.0f, 0.25f, 0.5f, 0.0f, 0.0f, 0.0f,
208
209     // Cara 3
210     0.0f, 0.5f, 0.0f, 0.25f, 0.0f, 0.0f, 0.0f, 0.0f,
211     0.5f, 0.0f, 0.0f, 0.5f, 0.25f, 0.0f, 0.0f, 0.0f,
212     0.0f, 0.0f, -0.5f, 0.0f, 0.25f, 0.0f, 0.0f, 0.0f,
213
214     // Cara 4
215     0.0f, 0.5f, 0.0f, 1.0f, 0.25f, 0.0f, 0.0f, 0.0f,
216     0.0f, 0.0f, -0.5f, 0.75f, 0.5f, 0.0f, 0.0f, 0.0f,
217     -0.5f, 0.0f, 0.0f, 0.5f, 0.25f, 0.0f, 0.0f, 0.0f,
218
219     // MITAD INFERIOR
220
221     // Cara 5
222     0.0f, -0.5f, 0.0f, 0.5f, 0.75f, 0.0f, 0.0f, 0.0f,
223     0.0f, 0.0f, 0.5f, 0.25f, 0.5f, 0.0f, 0.0f, 0.0f,
224     -0.5f, 0.0f, 0.0f, 0.75f, 0.5f, 0.0f, 0.0f, 0.0f,
225

```

Aquí guardo en variables los modelos y texturas antes declaradas.

```

263 camera = Camera(glm::vec3(0.0f, 0.0f, 0.0f), glm::vec3(0.0f, 1.0f, 0.0f), -60.0f, 0.0f, 0.3f, 0.5f);
264
265 brickTexture = Texture("Textures/brick.png");
266 brickTexture.LoadTextureA();
267 dirtTexture = Texture("Textures/dirt.png");
268 dirtTexture.LoadTextureA();
269 plainTexture = Texture("Textures/plain.png");
270 plainTexture.LoadTextureA();
271 pisoTexture = Texture("Textures/piso.tga");
272 pisoTexture.LoadTextureA();
273 dadoTexture = Texture("Textures/dadomeme.png");
274 dadoTexture.LoadTextureA();
275 cocheTexture = Texture("Textures/bob.png");
276 cocheTexture.LoadTextureA();
277 logofiTexture = Texture("Textures/escudo_fi_color.tga");
278 logofiTexture.LoadTextureA();
279
280
281 Kitt_M = Model();
282 Kitt_M.LoadModel("Models/Coche.fbx");
283 Parb = Model();
284 Parb.LoadModel("Models/CoPa.fbx");
285 Llanta_M = Model();
286 Llanta_M.LoadModel("Models/L1.fbx");
287 Llanta_M2 = Model();
288 Llanta_M2.LoadModel("Models/L2.fbx");
289 Llanta_M3 = Model();
290 Llanta_M3.LoadModel("Models/L3.fbx");
291 Llanta_M4 = Model();
292 Llanta_M4.LoadModel("Models/L4.fbx");

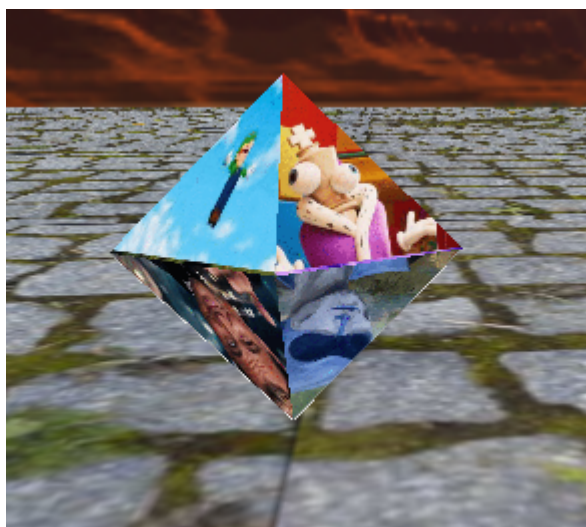
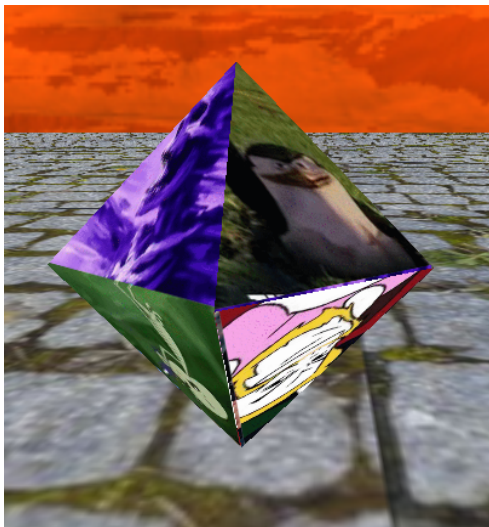
```

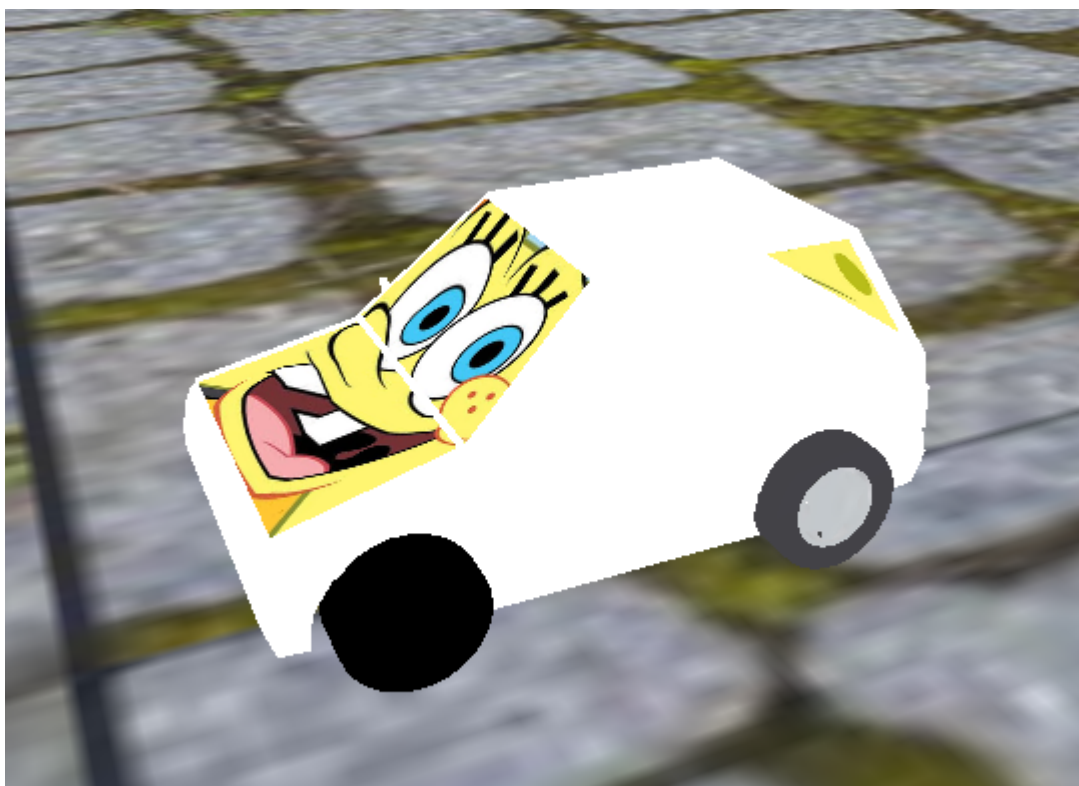
Llamo al modelo del dado de 8 caras.

```
//Dado de Opendgl
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-1.5f, 4.5f, -2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
dadoTexture.UseTexture();
meshList[4]->RenderMesh();
```

Aqui llamo las llantas así como las otras partes del coche además de acomodarlas en donde deben ir

```
387         cocheTexture.UseTexture();
388
389         model = glm::mat4(1.0);
390         model = glm::translate(model, glm::vec3(0.0f + mainWindow.getmuevex(), -0.5f, -3.0f));
391         model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
392         glm::mat4 modelBaseCoche = model;
393
394
395         glm::mat4 modelChasis = glm::scale(modelBaseCoche, glm::vec3(escalaCoche, escalaCoche, escalaCoche));
396
397         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelChasis));
398         Kitt.M.RenderModel();
399         Parb.RenderModel();
400
401         // --- LLANTAS ---
402
403         // Llanta delantera izquierda
404         model = modelBaseCoche;
405         model = glm::translate(model, glm::vec3(-1.23f, 1.35f, 0.5f));
406         model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
407         model = glm::scale(model, glm::vec3(escalaLlanta));
408         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
409         cocheTexture.UseTexture();
410         Llanta_M.RenderModel();
411
412         // Llanta trasera izquierda
413         model = modelBaseCoche;
414         model = glm::translate(model, glm::vec3(1.23f, 1.35f, 0.5f));
415         model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
416         model = glm::scale(model, glm::vec3(escalaLlanta));
417         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
418         cocheTexture.UseTexture();
419         Llanta_M2.RenderModel();
420
```





2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Tuve problemas para hacer el exportar el shader de las llantas por razones que desconozco me marcaba las llantas todas negras, pero tras forzar la textura solo 2 llantas aceptaron esta, las otras 2 no las usa y como se ve en el blender si tienen shaders



3.- Conclusión:

- a. Los ejercicios del reporte: Estuvieron entretenidos pues es curioso la forma en que se implementan los shaders además de crear el dado con memes fue divertido y fácil de implementar después.
- b. Comentarios generales: ninguno.
- c. Conclusión: Concluyo que es fundamental el uso y comportamiento de las texturas en diferentes entornos siendo el caso de OpenGL.

4.- Bibliografía en formato APA

de Vries, J. (s.f.). *Textures*. LearnOpenGL. Recuperado el 12 de abril de 2026, de <https://learnopengl.com/Getting-started/Textures>

de Vries, J. (s.f.). *Transformations*. LearnOpenGL. Recuperado el 12 de abril de 2026, de <https://learnopengl.com/Getting-started/Transformations>

G-Truc Creation. (s.f.). *OpenGL Mathematics (GLM): A C++ mathematics library for graphics programming*. Recuperado el 12 de abril de 2026, de <https://glm.g-truc.net/>